Comprehensive Technical Report: GenAI Predictive Maintenance in an Edge-Fog-Cloud Architecture

1. Executive Summary

This project implements a fully software-simulated, end-to-end industrial Internet of Things (IoT) ecosystem. Designed around the modern "Edge-Fog-Cloud" paradigm, the system moves beyond isolated machine learning notebooks to demonstrate how data dynamically travels from a physical factory floor, through localized routing gateways, up to a centralized cloud analytics platform.

The system emphasizes: - **Scalability via MQTT**: All telemetry is funneled through an asynchronous Message Queuing Telemetry Transport (MQTT) backbone. - **Bandwidth Optimization via Edge Computing**: Distributed intelligence sits on the "machines" themselves to catch anomalies before they consume cloud bandwidth. - **Explainable Generative AI**: Combines unsupervised anomaly detection (Isolation Forests) with Retrieval-Augmented Generation (RAG) to generate technical, citation-backed maintenance plans based on detected anomalies. - **Bi-Directional Management**: Features Over-The-Air (OTA) infrastructure, allowing administrators to push live configuration rules down to remote edge devices.

---

2. The Architectural Paradigm: Edge, Fog, and Cloud

Modern industrial applications generate gigabytes of sensor data per second. Sending all this raw data to a centralized cloud is entirely unfeasible due to bandwidth costs and latency. This project implements the three-tier hierarchy designed to solve this problem:

1. **The Edge Layer**: Represents the physical sensors or microcontrollers directly attached to the machinery. Devices at the Edge run localized, low-overhead algorithms to instantly detect critical violations (e.g., immediate temperature spikes) and trigger alerts in single-digit milliseconds.
2. **The Fog Layer**: Represents a local factory server or gateway. It gathers alerts from hundreds of local edge devices. It is responsible for localized enrichment—tagging hardware IDs with physical locations (e.g., "Rack 05") before passing the summarized state upward.
3. **The Cloud Layer**: The top echelon, responsible for mass data persistence, computationally expensive machine learning models, and Generative AI interaction. The cloud provides the top-down dashboard for human administration.

---

3. In-Depth Component Walkthrough

3.1 The Message Backbone (`docker-compose.yml` & `mosquitto.conf`)

At the center of the architecture is the **Eclipse Mosquitto** broker deployed via Docker. - **Role**: This broker handles the publish/subscribe (Pub/Sub) messaging pattern. It completely decouples the producers of data (the simulators) from the consumers (the edge agents and cloud databases). - **Configuration**: Exposed on port `1883`, the broker leverages a standard `mosquitto.conf` file allowing immediate sub/pub capabilities across multiple topics.

3.2 The Sensor Fleet Simulator (`scripts/mqtt_simulator.py`)

Because we lack physical hardware, this script synthetically generates our sensor fleet. - **Logic**: The script iterates over a fixed pool of devices (`device-01` to `device-05`). Every two seconds, it emits a telemetry snapshot (Temperature, Vibration, Current) randomized around a baseline using Gaussian (Normal) distributions. - **Anomaly Injection**: To ensure the ML algorithms and edge agents have something to catch, the script rolls a weighted random chance (`< 0.02`) to artificially inflate the temperature reading by up to 20 degrees. - **Networking**: Each JSON packet is fired securely into the MQTT broker under the topic `factory/<device_id>/telemetry`.

3.3 The Ingestion Engine & Data Store (`scripts/mqtt_ingest.py` & `src/db.py`)

The Cloud layer requires a persistent, historically accessible database for its Isolation models, instead of trying to capture UDP-like telemetry streams directly in the UI. - **`mqtt_ingest.py`**: A continuously running headless daemon. It subscribes broadly to the wildcard topic `factory/+/telemetry`. Upon catching a packet, it translates the JSON into a Pandas DataFrame format. - **`src/db.py`**: The ingestor hands the dataframe to `db.py`, which leverages Python's native SQLite3 library (a highly robust local database engine). Data is appended dynamically into a structured `telemetry` table featuring indexes on both the `device_id` and the `timestamp` to ensure the stream queries from the `app.py` dashboard operate in optimal `O(log N)` time limits.

3.4 The Edge Node Intelligence (`src/edge_agent.py`)

This script models the constrained environment of a physical microcontroller deployed in the field. - **Telemetry Sniffing**: The agent listens directly to the raw telemetry firehose. - **Local Threshold Processing**: For every single tick of data, the agent compares the payload against its internal static variables (`THRESHOLDS`). If a `device-04` packet comes in at 67°C, and the threshold is 60°C, the agent identifies it immediately. - **Alert Generation**: Rather than sending a massive batch of raw data, the agent constructs a lightweight, compressed `critical` or `warning` JSON alert identifying *why* it tripped. It publishes this

to a secondary topic: `factory/<device_id>/alerts`. - **OTA Command Listening**: Crucially, the edge agent also subscribes to `factory/+/cmd/firmware`. When a configuration packet drops, the agent dynamically overwrites its internal `THRESHOLDS` dictionary on the fly without ever restarting its execution loop.

3.5 The Fog Gateway Hub (`fog_app.py`)

Sitting between the Edge and the Cloud is the Fog Gateway—designed locally for a factory floor manager. - **Implementation**: It is built as a secondary, lightweight Streamlit dashboard running on port `8502`. - **Background Multithreading**: The magic of the Fog Gateway is that it spins up an asynchronous MQTT client thread in the background utilizing `@st.cache_resource`. This thread perpetually listens to edge alerts. - **Enrichment**: When an alert drops, the Fog logic appends contextual string data (e.g., translating `device-03` to `Factory-Floor-A (Rack 03)`). It then caches these enriched alerts locally into `data/fog_alerts.jsonl` allowing both the local Fog UI and the global Cloud UI to read them natively.

3.6 The Central Cloud Analytics Platform (`app.py`)

The primary `app.py` file is a massive, multi-tab Streamlit web application. This acts as our Central Cloud Analytics Tier.

Tab 1: Data Source Selection - The UI allows administrators to toggle between historical static datasets (NASA C-MAPSS) and the new "Live MQTT (SQLite)" pipeline. Selecting the Live pipeline redirects all dataframe operations to query the `telemetry.db` SQLite engine.

Tab 2: The Core Dashboard and Anomaly Modeling (`src/anomaly.py`) - The Dashboard pulls the thousands of historical records stored by the ingestor. - It passes this historical dataframe into an **Isolation Forest** (an unsupervised machine-learning algorithm built on `scikit-learn`). - **The specific mechanism**: Because we have multiple sensors drifting concurrently, simple "temp > 50" logic is insufficient. The Isolation forest isolates multidimensional outliers. We reverse the `decision_function` array to generate an `anomaly_score`. - A dynamic line chart renders this score overlaid against the P90 and P98 severity percentiles.

Tab 3: Generative RAG Explainability (`src/rag.py` & `src/llm.py`) - When the ML model flags anomalous behaviour, generating an alert is often not enough for a field engineer. The dashboard utilizes Retrieval-Augmented Generation. - Users can upload raw PDFs of operating manuals directly into the `docs/` folder. The `build_index` function chunks this text and converts it into embeddings via Sentence-Transformers, loading it into a High-Dimensional Vector Database (FAISS). - When an engineer clicks an anomaly in the UI, `app.py` injects the specific context of that machine failure into a hardcoded prompt skeleton. It queries the FAISS vector database for the top-K relevant manual chunks and hands the whole package to an LLM (Ollama). - The output

is a highly structured, directly actionable Work Order explicitly referencing citation snippets for immediate maintenance dispatch.

Tab 4 & 5: Bi-Directional Cloud Command (The OTA Simulator) - The Cloud platform doesn't just read data; it controls the edge. - In the "OTA Updates" tab, an administrator enters new global safety thresholds (e.g., dropping the Temp limit from 60°C to 50°C because it is summer). - The UI generates a versioned JSON packet (cryptographically hashed using `hashlib.sha256` to simulate payload security) and publishes it down the MQTT pipe to the `cmd/firmware` topic. - Because `edge_agent.py` is actively listening to this pipeline, it catches the command and enforces the new sensitivity dynamically, bridging the gap between Cloud command and Edge execution seamlessly.

---

4. Closing Thoughts

By synthesizing open-source tools (Docker, Mosquitto, Streamlit, Scikit-Learn, FAISS, Ollama) against strict architectural standards, this platform successfully mirrors a highly advanced enterprise IoT solution. It effectively demonstrates that the future of predictive maintenance lies in combining localized heuristics (Edge processing) with global pattern recognition (Cloud ML) and automated reasoning (Generative AI).